MATH 8090: Spectral Analysis of Time Series I

Whitney Huang, Clemson University

10/26-28/2026

Contents

Background . . . 1
An example of cosine wave . . . 1
An example from Cryer and Chan Cryer and Chan (2008), Chapter 13 . . . 3
Illustration of aliasing . . . 5
Periodogram . . . 6
Toy Examples . . . 6
Example 4.3 from Shumway and Stoffer (2017) . . . 13
Spectral Density . . . 14
Spectral Density of an MA(1) Process with $\theta = \pm 0.5$ . . . 14
Spectral Density of an AR(1) Process with $\phi = \pm 0.9$ . . . 15
Spectral Density of an AR(2) Process with $\phi_1 = 1.5$ and $\phi_2 = -0.95$ . . . 16
Spectral Density of a Seasonal AR Model . . . 17
Spectral Estimation . . . 18
The Periodogram Is Not a Consistent Estimator . . . 18
Averaged Periodogram . . . 20
Pointwise Confidence Intervals for $f(\omega)$ . . . 21
Leakage . . . 23
Tapering . . . 24
References . . . 26

Background

An example of cosine wave

A cosine wave can be written as

$$y(t) = A\cos(2\pi\omega t + \phi),$$

where A is the amplitude, ω is the frequency in cycles per unit time, and ϕ is the phase in radians. The period is $1/\omega$, and the horizontal shift is $-\phi/(2\pi\omega)$.

```r
source("curlyBraces.R")
# Set the amplitude, frequency, and phase.
A <- 2
omega <- 1
phi <- 1
shift <- -phi / (2 * pi * omega)
period <- 1 / omega

t <- seq(shift, 3, length.out = 200)
y <- A * cos(2 * pi * omega * t + phi)

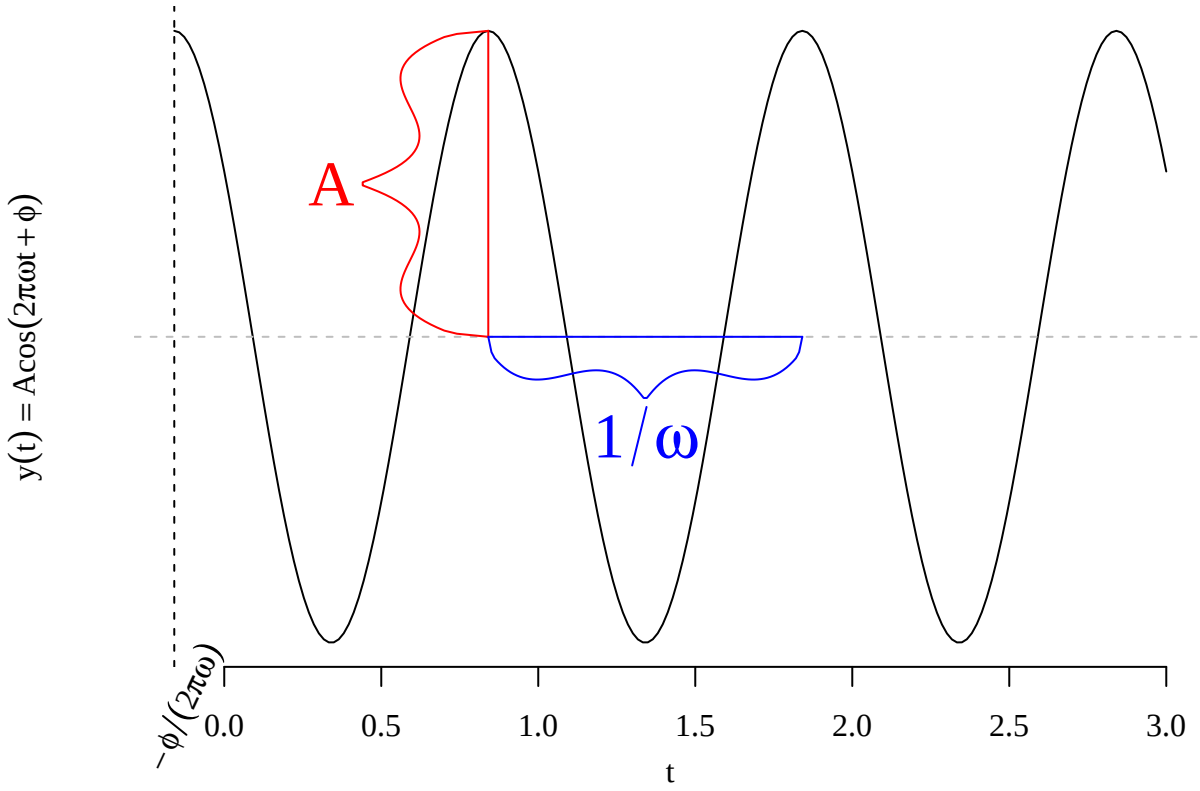
par(mgp = c(2.2, 1, 0), mar = c(4.5, 4, 0.8, 0.6), las = 1, family = "serif")
fu <- expression(y(t) == A * cos(2 * pi * omega * t + phi))
plot(t, y, type = "l", yaxt = "n", bty = "n", ylab = fu)
abline(h = 0, lty = 2, col = "gray")

# Mark the amplitude and one complete cycle.
peak <- shift + period
segments(peak, 0, peak + period, 0, col = "blue")
segments(peak, 0, peak, A, col = "red")

CurlyBraces(peak, 0.5 * A, A, pos = 1, direction = 2, col = "red")
text(peak - 0.5 * period, 0.5 * A, "A", family = "serif", col = "red", cex = 2)

CurlyBraces(0, peak + 0.5 * period, period, pos = 2, direction = 2, col = "blue")
text(peak + 0.5 * period, -0.65, expression(1 / omega), family = "serif",
     col = "blue", cex = 2)

# Indicate the horizontal shift due to the phase.
abline(v = shift, lty = 2)
text(shift, par("usr")[3] - 0.1, labels = expression(-phi / (2 * pi * omega)),
     srt = 65, pos = 1, xpd = TRUE)
```



An example from Cryer and Chan Cryer and Chan (2008), Chapter 13

Consider a series formed by adding two cosine waves:

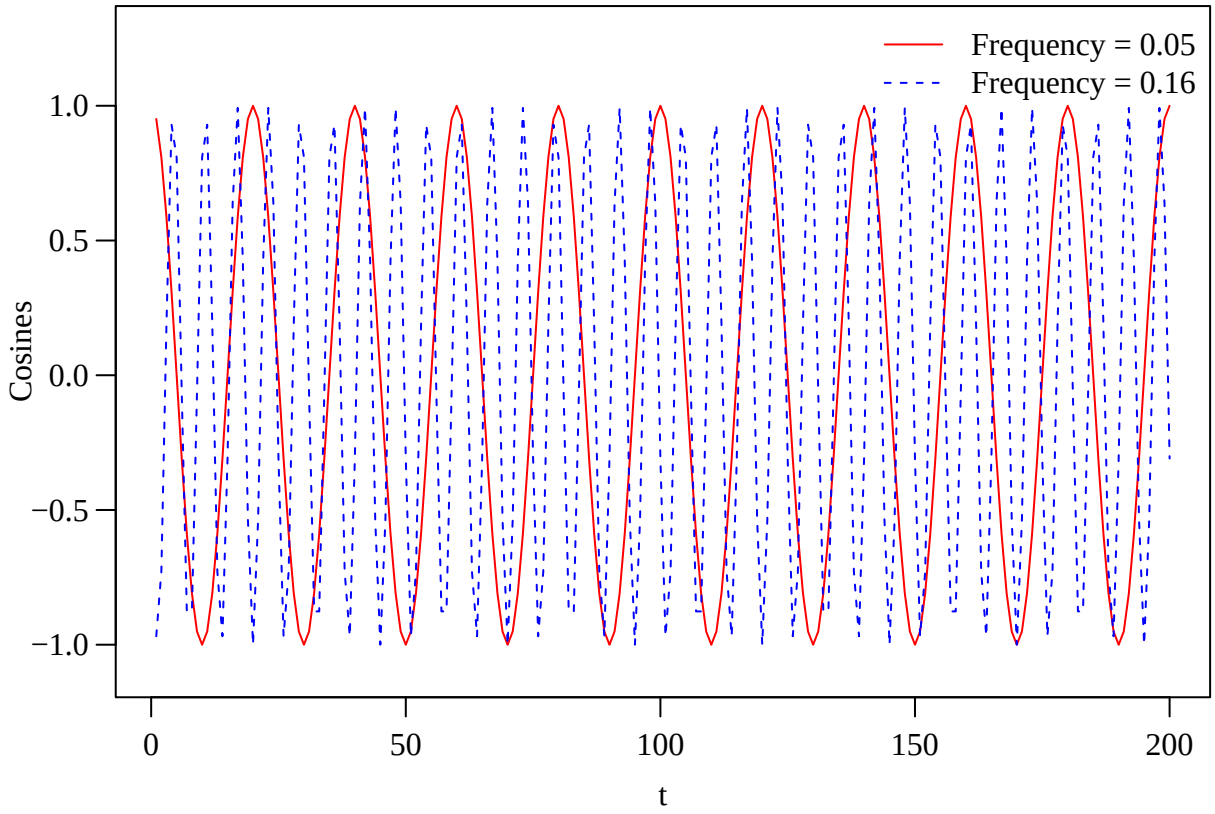
$$y_t = 3\cos\left(2\pi\frac{10}{200}t\right) + 2\cos\left(2\pi\left(\frac{32}{200}t + 0.3\right)\right).$$

The frequencies are 0.05 and 0.16 cycles per observation, corresponding to periods of 20 and 6.25 observations. The term 0.3 specifies a phase of 0.3 cycles, or 0.6π radians, for the second wave. First, we plot the two components with unit amplitude to compare their oscillations.

```
t <- seq(1, 200, len = 200)

# Construct the two unit-amplitude cosine waves.
cos1 <- cos(2 * pi * t * 10 / 200)
cos2 <- cos(2 * pi * (t * 32 / 200 + 0.3))

par(mar = c(3.5, 3.5, 1, 0.5), mgp = c(2, 0.7, 0), las = 1, family = "serif")
plot(t, cos1, ylab = "Cosines", col = "red", type = "l",, ylim = c(-1.1, 1.275))
lines(t, cos2, col = "blue", lty = 2)
legend("topright", legend = c("Frequency = 0.05", "Frequency = 0.16"),
       col = c("red", "blue"), lty = c(1, 2), bty = "n")
```



Next, we combine the waves with amplitudes 3 and 2. The time plot shows their combined pattern, while the ACF reflects the recurring dependence. The periodogram separates the two oscillations into peaks at their respective frequencies; the larger peak corresponds to the component with the larger amplitude.

```
library(astsa)
library(TSA)

# Combine the components with their specified amplitudes.
y <- 3 * cos1 + 2 * cos2

par(mgp = c(2.7, 1, 0), mar = c(3.5, 4.5, 0.8, 0.6), las = 1, mfrow = c(3, 1),
    family = "serif")

astsa::tsplot(y, ylab = expression(y[t]))
acf(y)

# Identify the two generating frequencies in the periodogram.
TSA::periodogram(y, ylab = "")
abline(h = 0)
abline(v = c(10 / 200, 32 / 200), lty = 2, col = "gray", lwd = 2)
axis(1, at = c(10 / 200, 32 / 200))
```

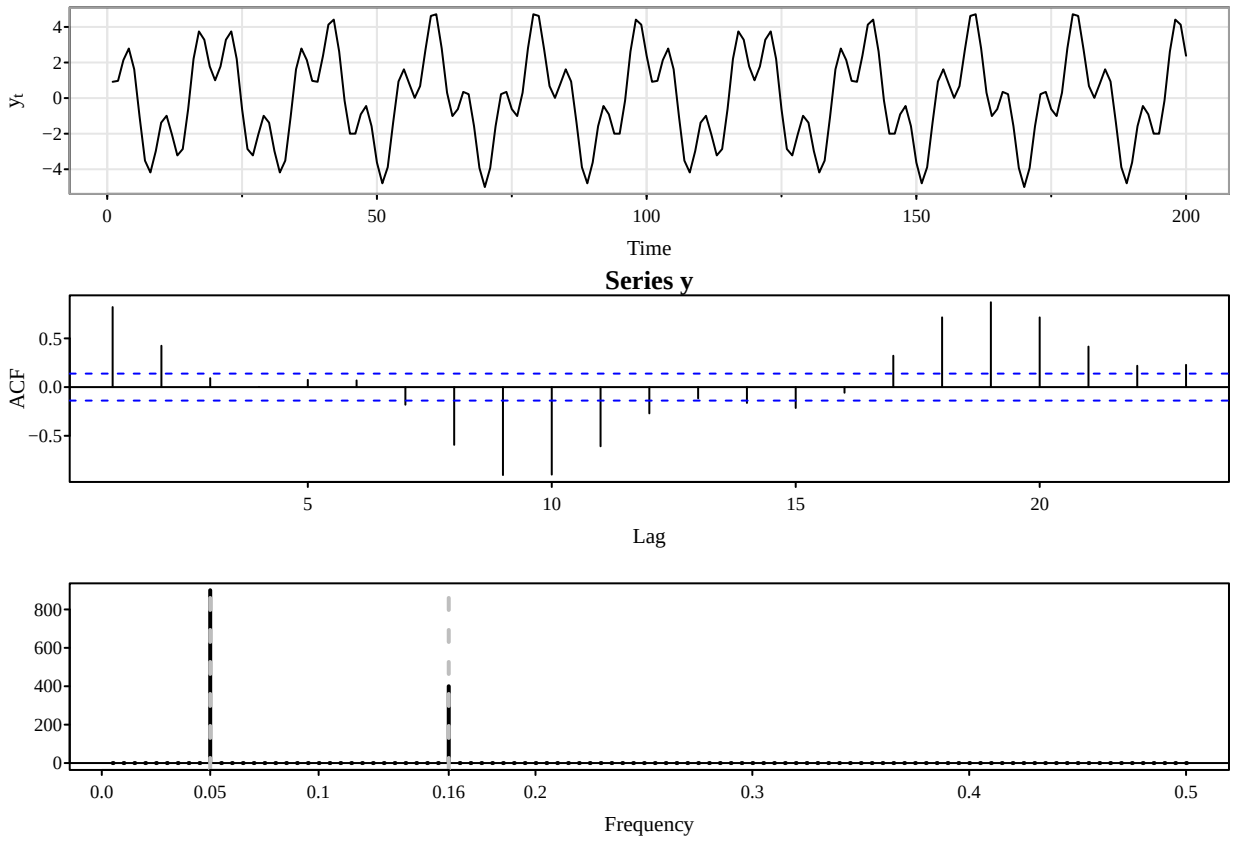


Illustration of aliasing

Aliasing occurs when different continuous-time oscillations produce the same sampled observations. Here, cosine waves with frequencies 0.25 and 0.75 cycles per unit time agree at every integer sampling time, even though they differ between those times.

With observations spaced one unit apart, the Nyquist frequency is 0.5 cycles per unit time. The frequency 0.75 aliases to 0.25, so these sampled values alone cannot distinguish the two waves.

```
par(mgp = c(2, 0.7, 0), mar = c(3.5, 4, 0.8, 0.6), las = 1, family = "serif")

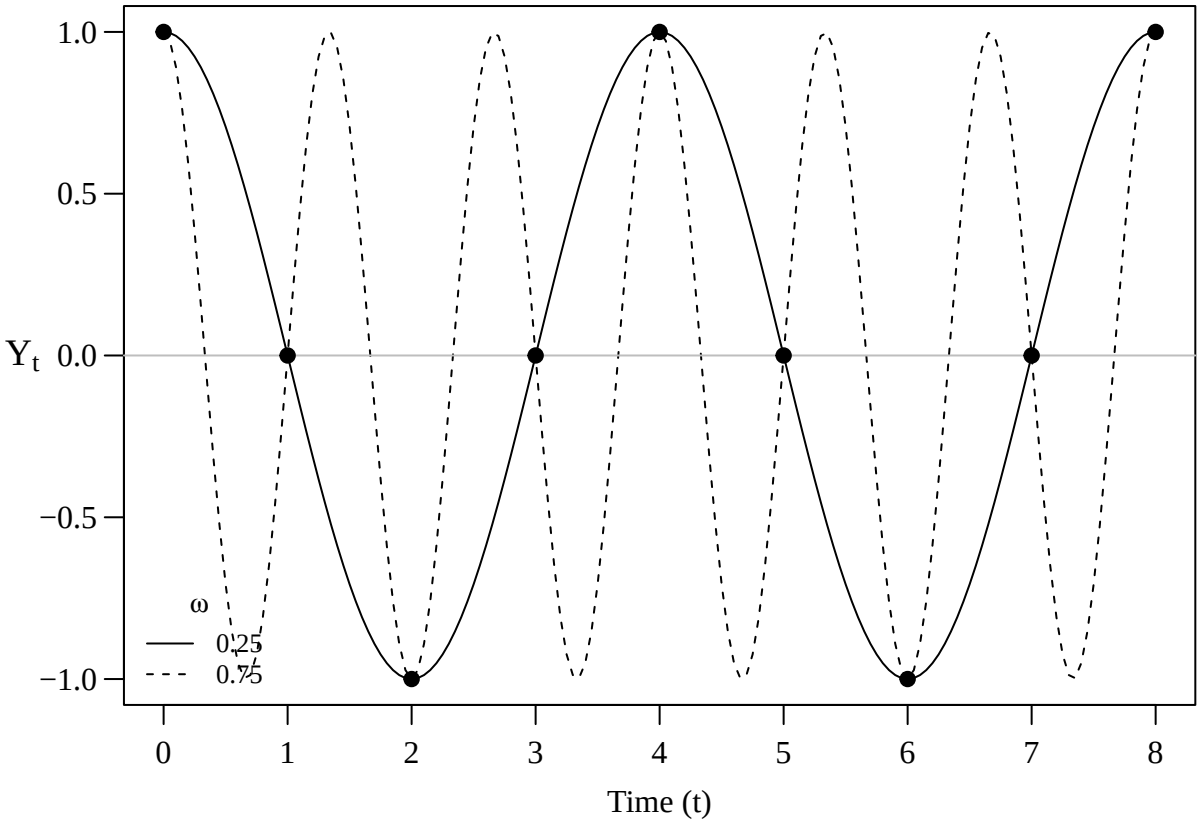
# Use a fine grid to display the underlying continuous curves.
t <- seq(0, 8, by = 0.05)
plot(t, cos(2 * pi * t / 4), axes = FALSE, type = "l", ylab = "", xlab = "Time (t)")
mtext(expression(Y[t]), 2, las = 1, line = 2.2, cex = 1.2)
axis(1, at = 0:8)
axis(2)
box()

lines(t, cos(2 * pi * t * 3 / 4), lty = "dashed")
abline(h = 0, col = "gray")

# Both curves pass through the same integer-time observations.
points(0:8, cos(2 * pi * (0:8) / 4), pch = 19)

legend("bottomleft", legend = c("0.25", "0.75"),
```

```
       title = expression(omega), lty = c(1, 2),
       bty = "n", cex = 0.8)
```



Periodogram

The periodogram describes how the variation in an observed series is distributed across frequencies. For observations spaced one unit apart, frequencies are measured in cycles per observation. A peak at frequency ω corresponds to a period of $1/\omega$ observations.

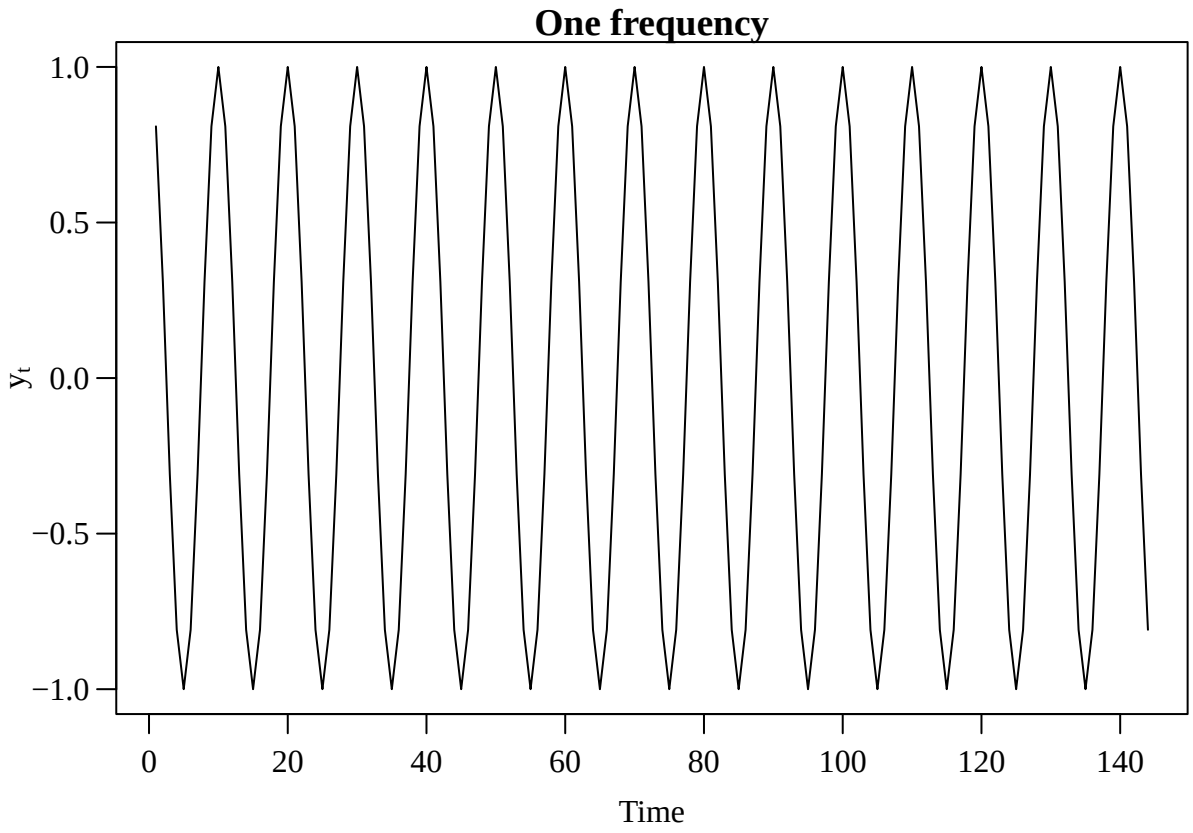
Toy Examples

We begin with a cosine wave at frequency 0.1, then add a second wave at frequency 0.234, and finally add white noise. The periodograms show how these changes affect the frequency representation.

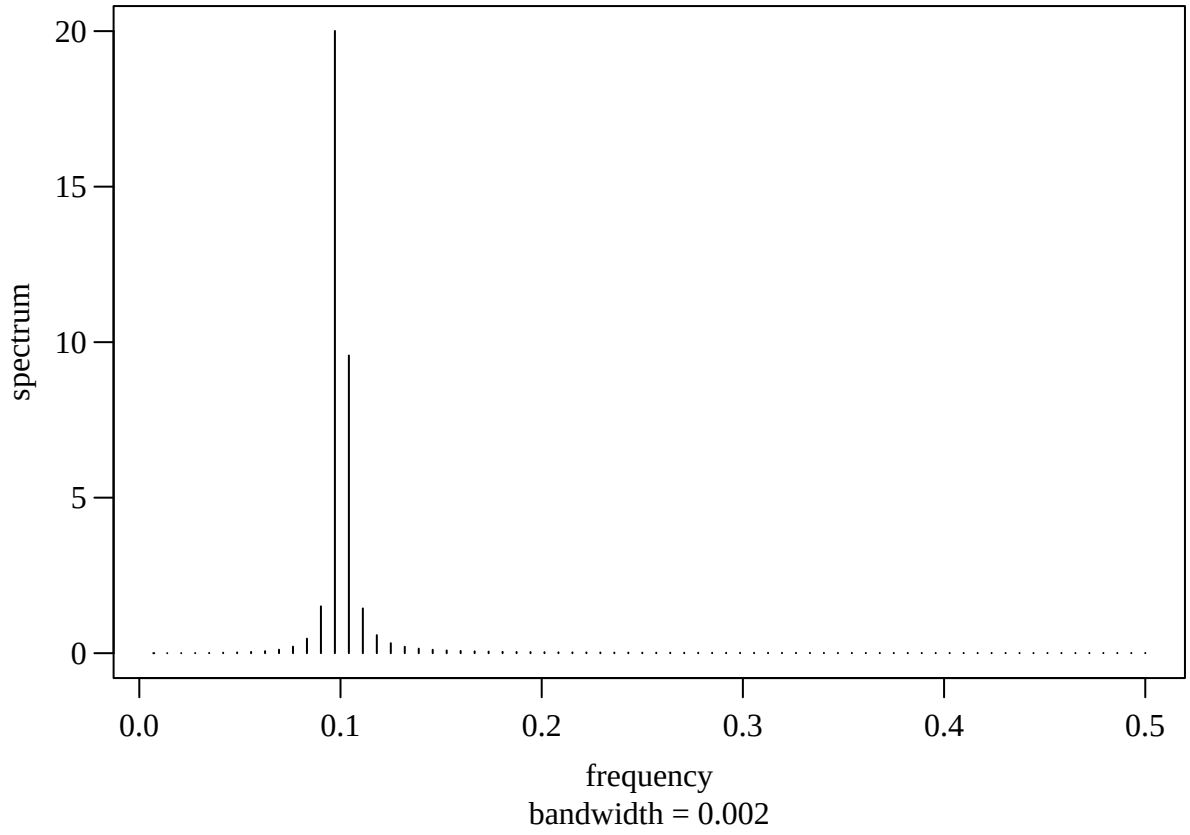
```
set.seed(123)
n <- 144
t <- seq_len(n)

par(las = 1, mgp = c(2, 0.7, 0), mar = c(4, 4, 1, 0.6), family = "serif")

# Start with one oscillation: period = 10 observations.
y <- cos(2 * pi * 0.1 * t)
ts.plot(y, ylab = expression(y[t]), main = "One frequency")
```




```
spectrum(y, log = "no", main = "", type = "h", taper = 0, detrend = FALSE, demean = TRUE)
```



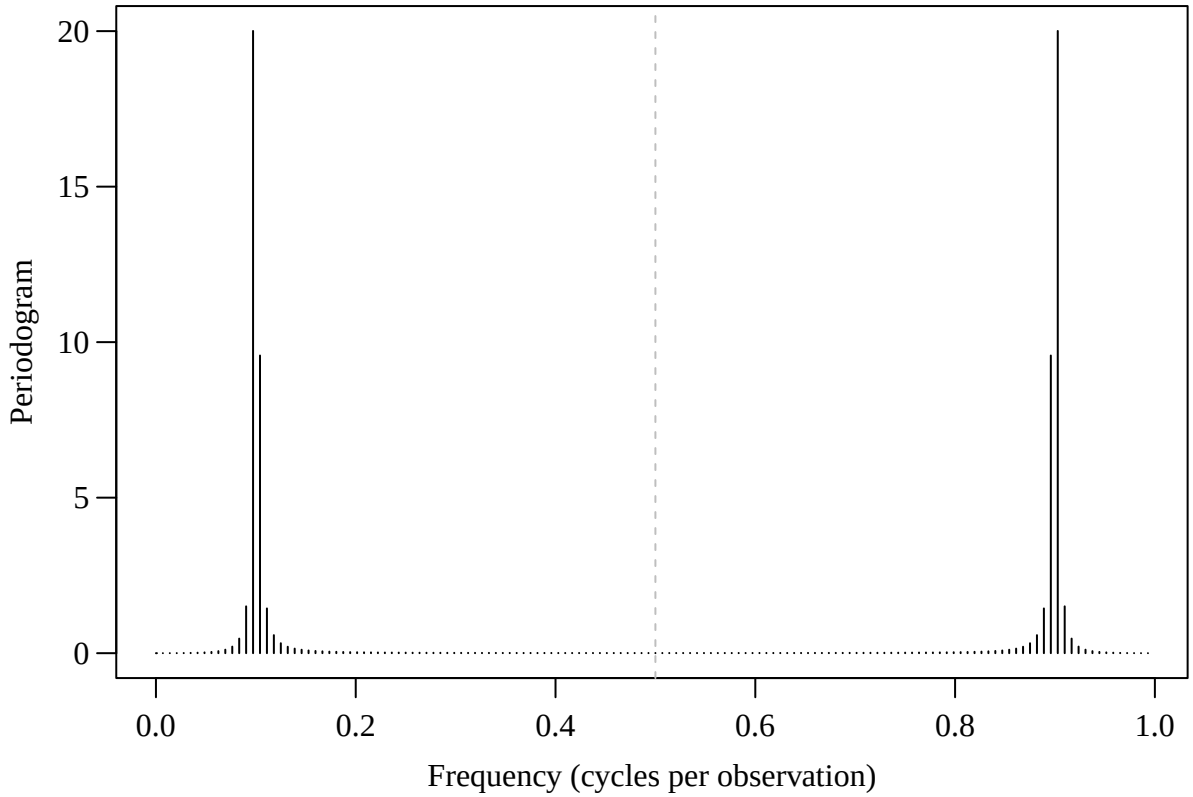
We can also compute the periodogram directly using the fast Fourier transform (FFT):

$$I_n(\omega_j) = \frac{1}{n}\left|\sum_{t=1}^{n}(y_t - \bar{y})e^{-2\pi i\omega_j t}\right|^2, \qquad \omega_j = \frac{j}{n}.$$

For a real-valued series, the FFT power is symmetric: frequencies ω and $1-\omega$ contain the same information. Thus, we usually display only frequencies up to the Nyquist frequency, 0.5.

```r
# Compute FFT power over the full frequency grid.
Per <- Mod(fft(y - mean(y)))^2 / n
Freq <- (0:(n - 1)) / n

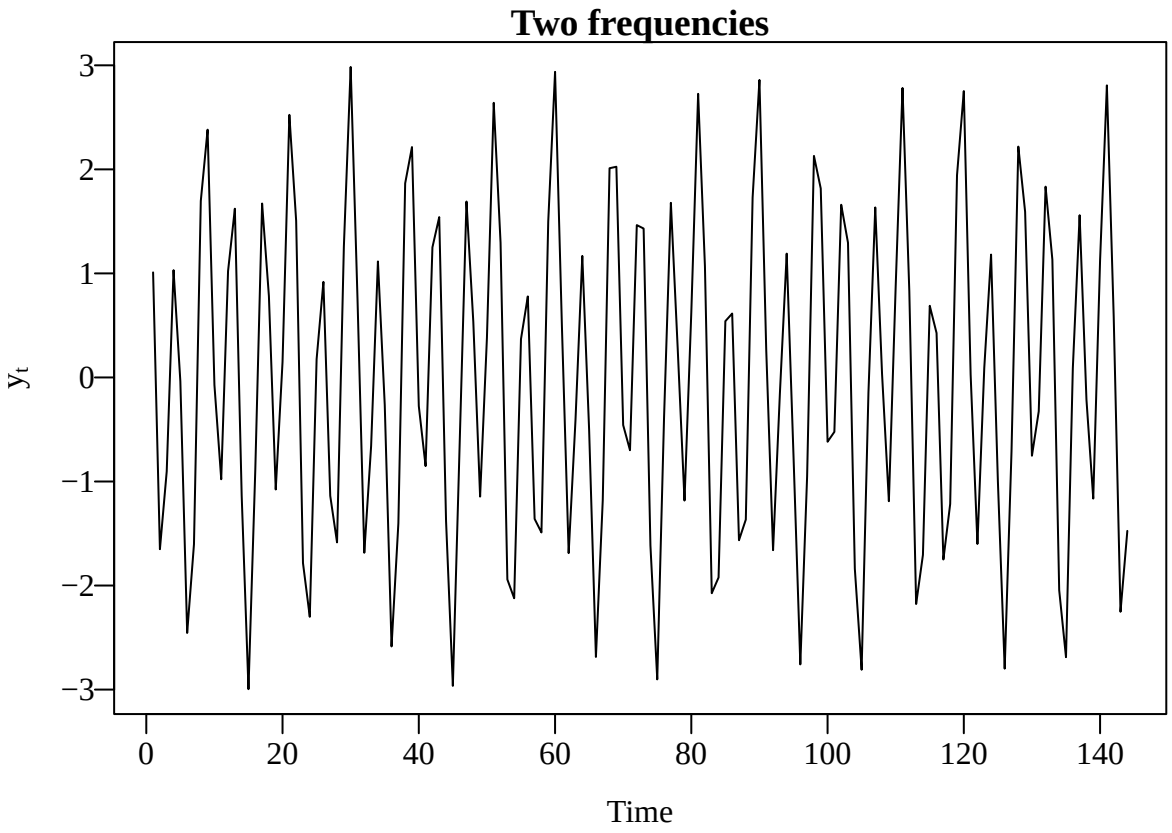
par(las = 1, mgp = c(2, 0.7, 0), mar = c(4, 4, 1, 0.6), family = "serif")
plot(Freq, Per, type = "h", xlab = "Frequency (cycles per observation)", ylab = "Periodogram")
abline(v = 0.5, lty = 2, col = "gray")
```



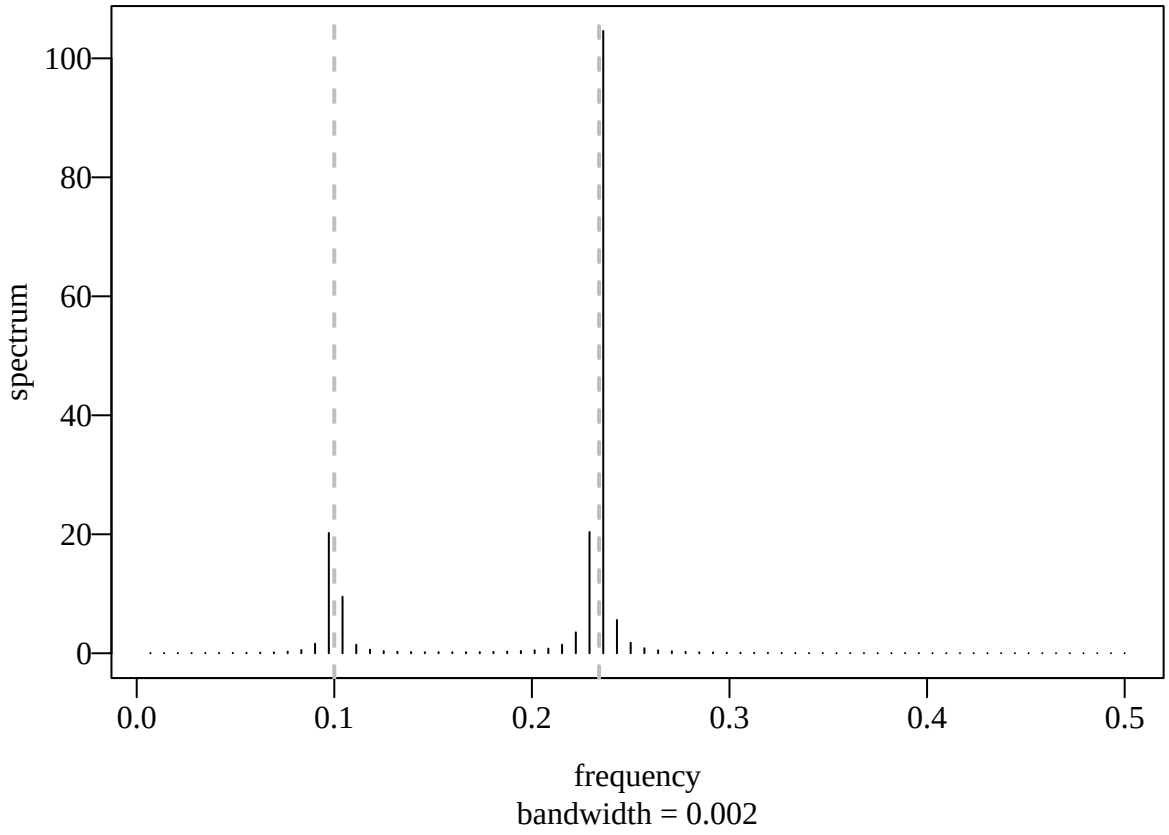
The generating frequencies do not fall exactly on the Fourier grid j/n. Consequently, their power spreads across neighboring frequencies, illustrating spectral leakage.

```
# Add a second oscillation with a larger amplitude.
y <- y + 2 * cos(2 * pi * 0.234 * t)

par(las = 1, mgp = c(2, 0.5, 0), mar = c(4, 4.5, 1, 0.6), family = "serif")
ts.plot(y, ylab = expression(y[t]), main = "Two frequencies")
```

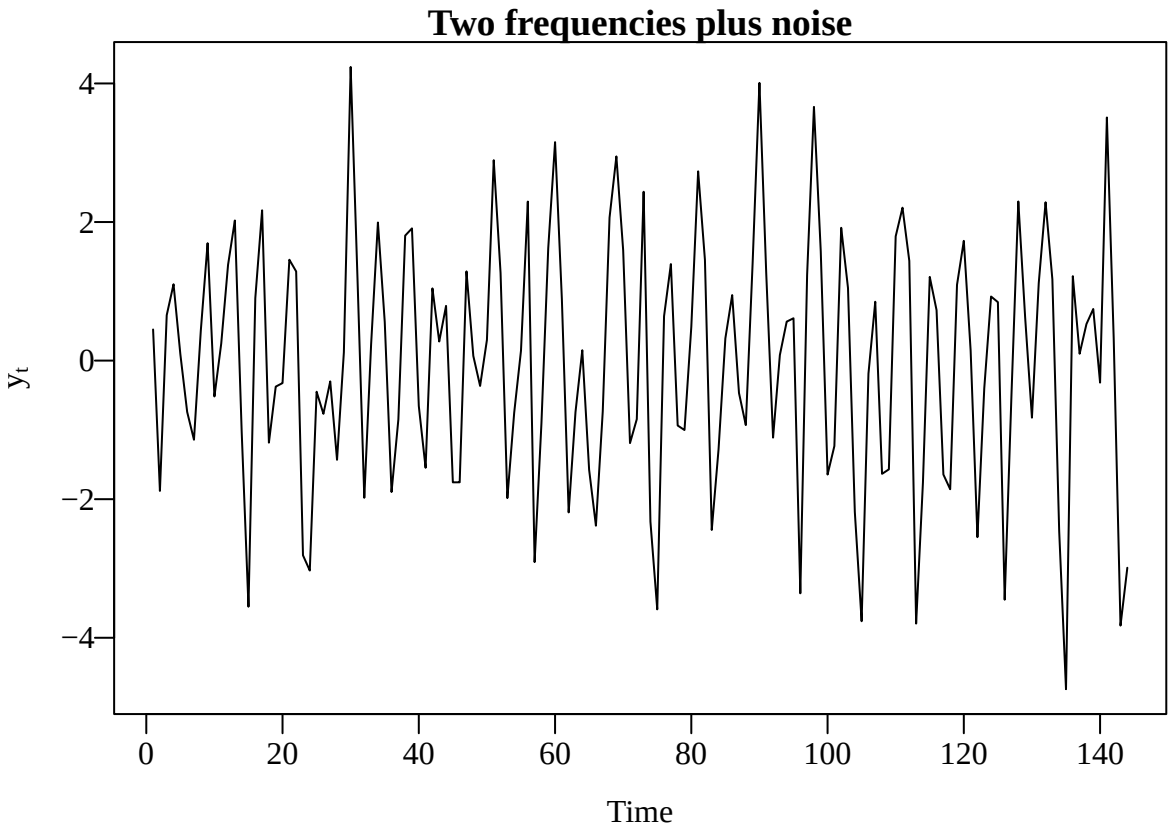



```
spectrum(y, log = "no", main = "", type = "h", taper = 0, detrend = FALSE, demean = TRUE)
abline(v = c(0.1, 0.234), lty = 2, col = "gray", lwd = 2)
```

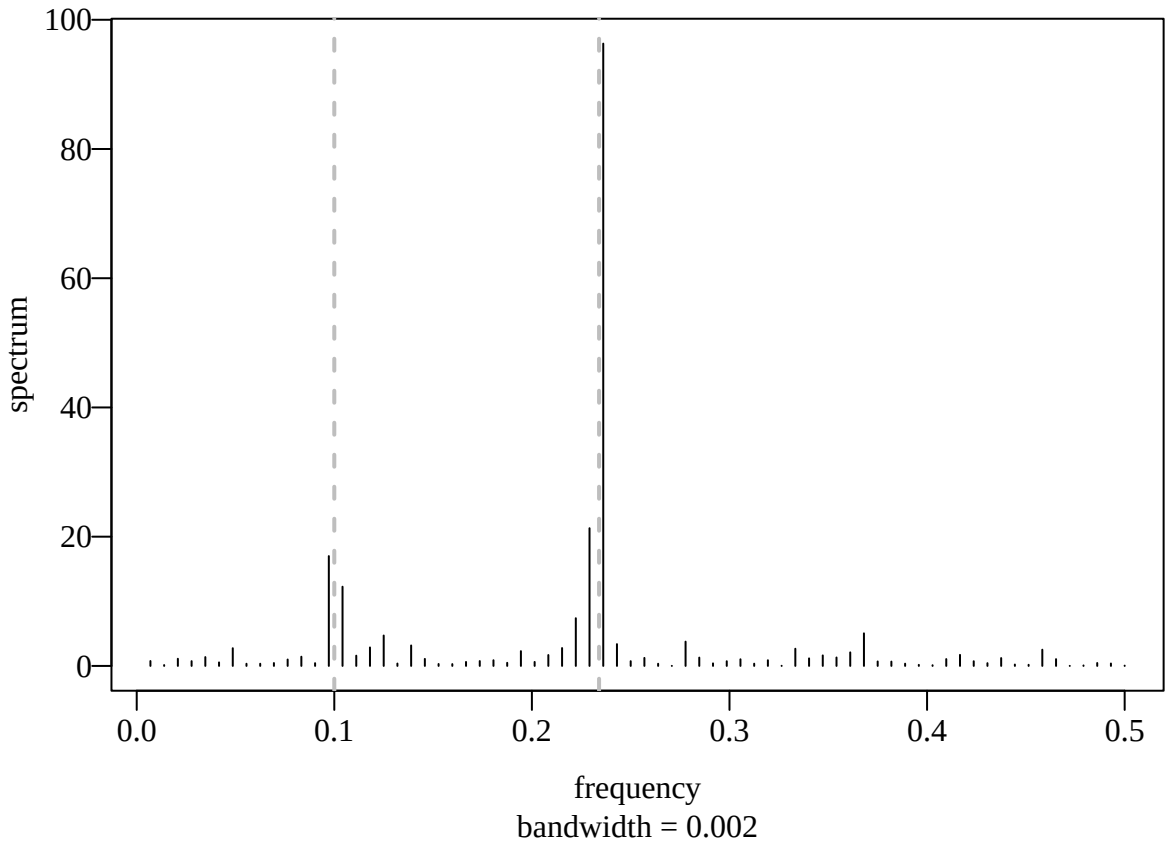



```
# Add white noise, which contributes power across frequencies.
y <- y + rnorm(n)

ts.plot(y, ylab = expression(y[t]), main = "Two frequencies plus noise")
```




```
spectrum(y, log = "no", main = "", type = "h", taper = 0, detrend = FALSE, demean = TRUE)
abline(v = c(0.1, 0.234), lty = 2, col = "gray", lwd = 2)
```



The stronger oscillation produces the larger spectral peak. Adding noise makes the periodogram more irregular, although both oscillatory components remain visible.

Example 4.3 from Shumway and Stoffer (2017)

The `star` series records the magnitude of a variable star. Its periodogram helps identify recurring cycles that may be difficult to separate in the time plot. We focus on low frequencies to examine cycles lasting approximately 24–29 days.

```
library(astsa)

n <- length(star)
par(mfrow = c(2, 1), las = 1, mgp = c(2, 0.7, 0), mar = c(4, 4.5, 1, 0.6))

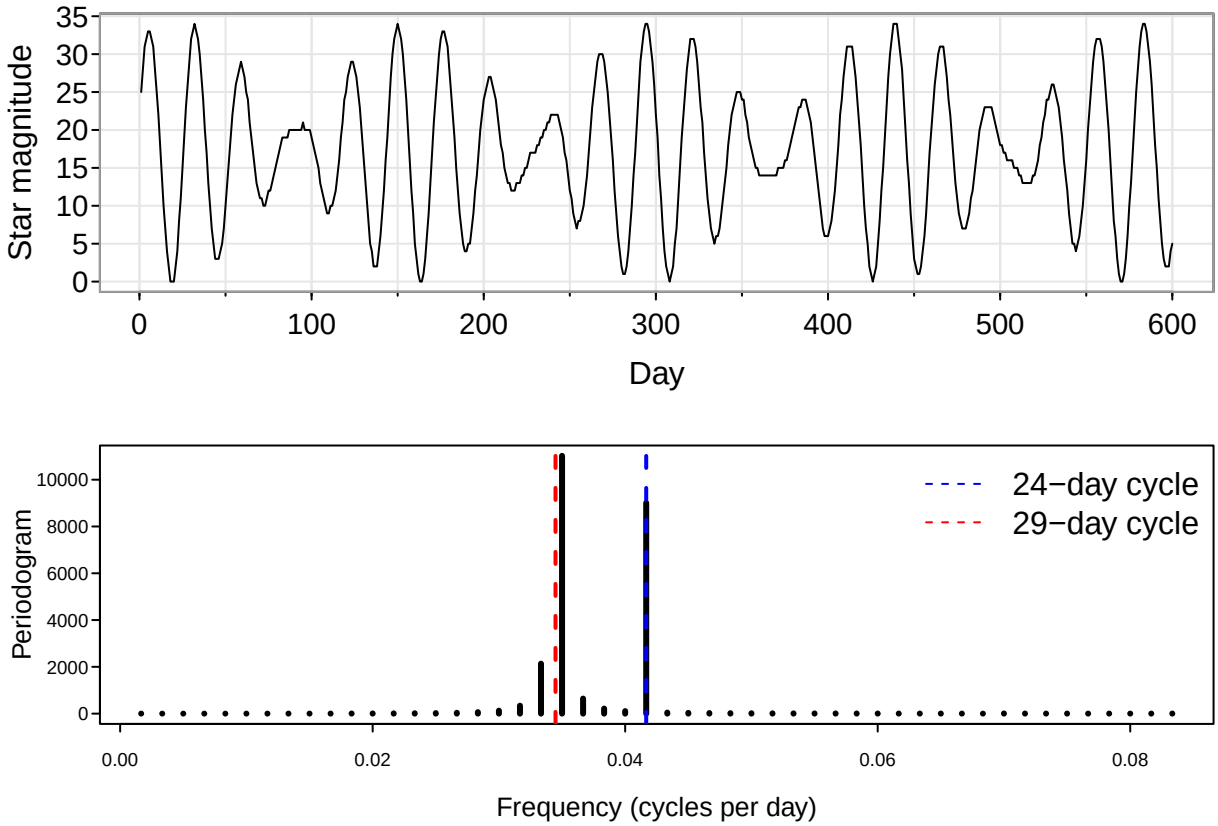
astsa::tsplot(star, ylab = "Star magnitude", xlab = "Day")

# The first FFT entry is frequency zero; omit it from this plot.
Per <- Mod(fft(star - mean(star)))^2 / n
Freq <- (0:(n - 1)) / n
keep <- seq_len(min(50, floor(n / 2))) + 1

plot(Freq[keep], Per[keep], type = "h", lwd = 3,
     ylab = "Periodogram", xlab = "Frequency (cycles per day)",
     cex.axis = 0.6, cex.lab = 0.8)

# Translate the reference cycle lengths into frequencies.
```

```r
abline(v = c(1 / 24, 1 / 29), lty = 2, col = c("blue", "red"), lwd = 2)
legend("topright", legend = c("24-day cycle", "29-day cycle"),
       col = c("blue", "red"), lty = 2, bty = "n")
```



Spectral Density

The spectral density describes the population distribution of variance across frequencies for a stationary process. In contrast, the periodogram is calculated from a finite observed series and can be highly variable.

The following examples compare theoretical spectra with simulated realizations. Each realization is only one possible series from the specified model.

Spectral Density of an MA(1) Process with $\theta = \pm 0.5$

Using the convention

$$Y_t = \varepsilon_t + \theta \varepsilon_{t-1},$$

the spectral density is

$$f(\omega) = \sigma^2 \left[1 + \theta^2 + 2\theta \cos(2\pi\omega)\right].$$

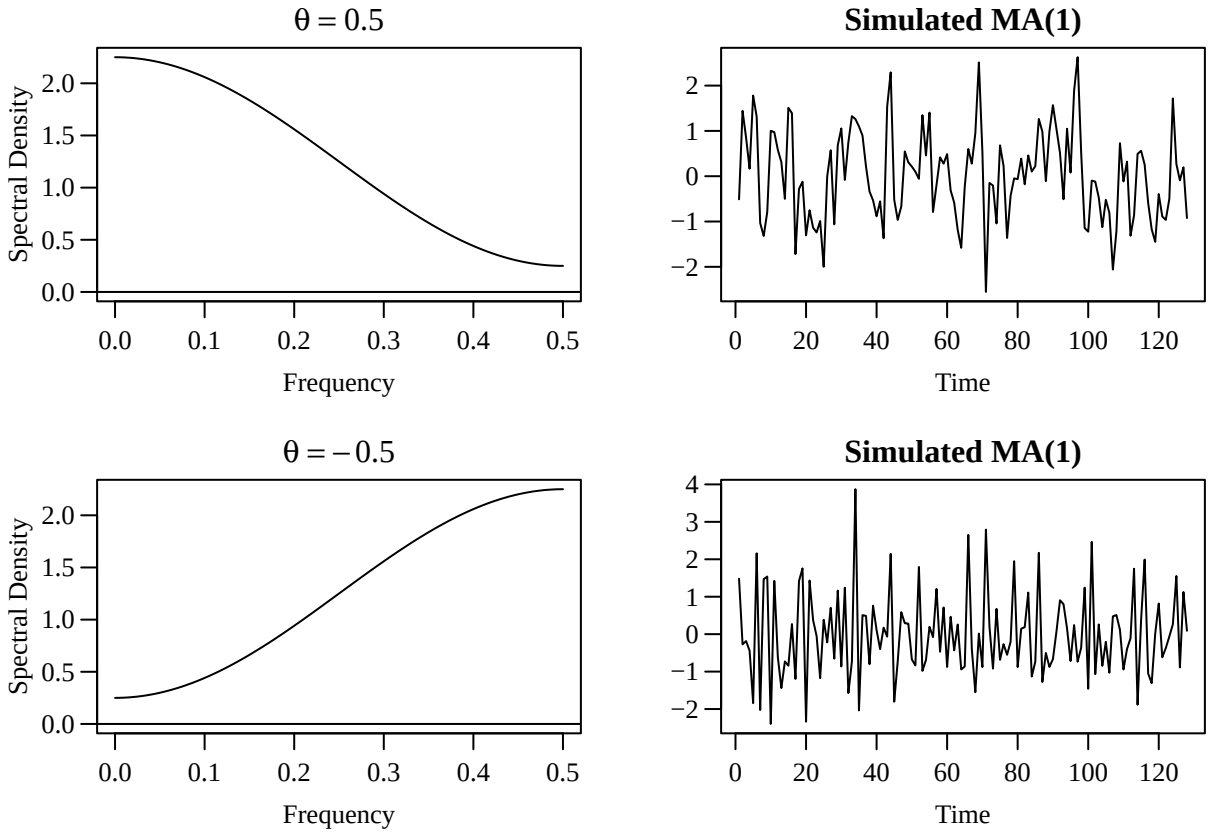
A positive θ emphasizes low frequencies and produces positive lag-one correlation. A negative θ emphasizes high frequencies and produces negative lag-one correlation.

```r
set.seed(123)
theta <- 0.5
```

```r
par(las = 1, mar = c(3.8, 3.8, 1.8, 0.6), mfrow = c(2, 2), mgp = c(2, 0.7, 0),
    family = "serif")

# Positive MA coefficient: more power at low frequencies.
TSA::ARMAspec(model = list(ma = theta))
title(main = expression(theta == 0.5))
ts.plot(arima.sim(n = 128, model = list(ma = theta)), ylab = "", main = "Simulated MA(1)")

# Negative MA coefficient: more power at high frequencies.
TSA::ARMAspec(model = list(ma = -theta))
title(main = expression(theta == -0.5))
ts.plot(arima.sim(n = 128, model = list(ma = -theta)), ylab = "", main = "Simulated MA(1)")
```



Spectral Density of an AR(1) Process with $\phi = \pm 0.9$

For the stationary AR(1) model

$$Y_t = \phi Y_{t-1} + \varepsilon_t,$$

the spectral density is

$$f(\omega) = \frac{\sigma^2}{1 + \phi^2 - 2\phi\cos(2\pi\omega)}.$$

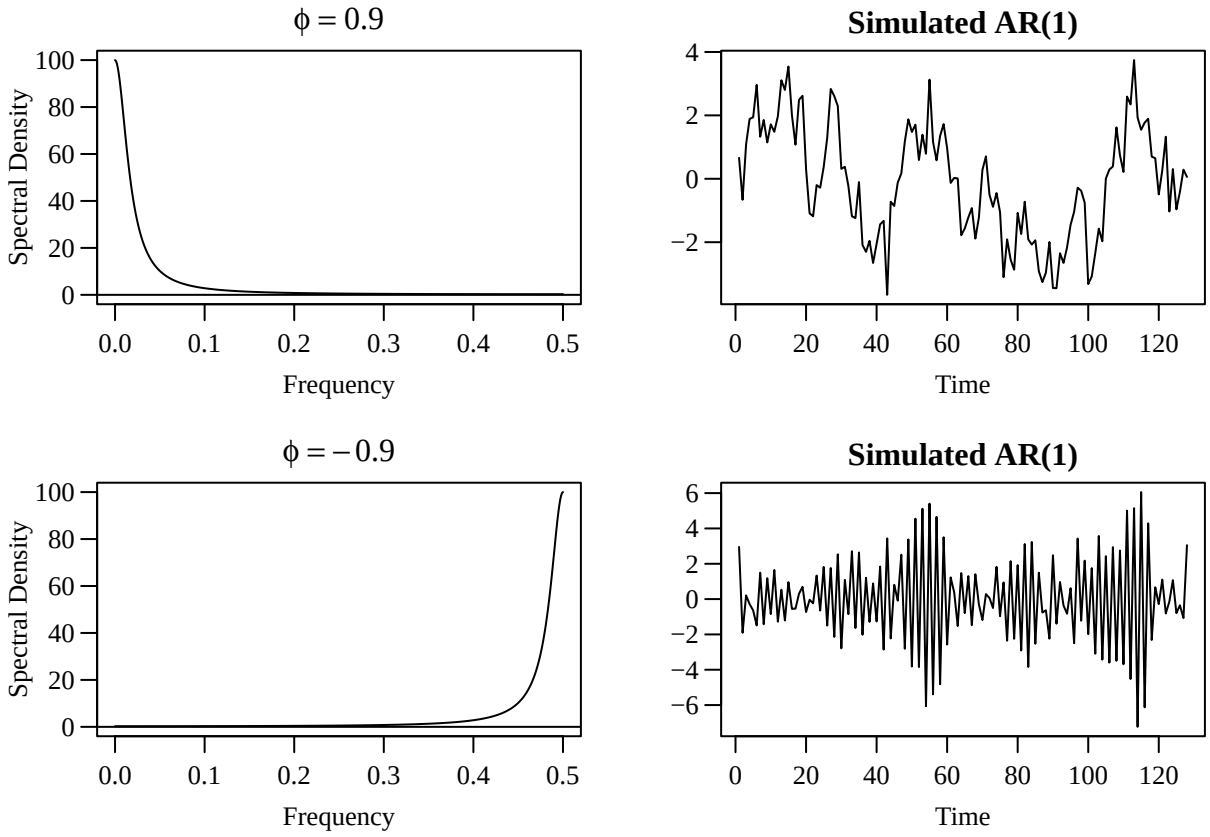
When $\phi = 0.9$, power concentrates near frequency zero, producing persistent, slowly varying behavior. When $\phi = -0.9$, power concentrates near frequency 0.5, producing frequent alternation between positive and negative values.

```r
phi <- 0.9

par(las = 1, mar = c(3.8, 3.8, 1.8, 0.6), mfrow = c(2, 2), mgp = c(2, 0.7, 0),
    family = "serif")

# Positive AR coefficient: persistent, slowly varying behavior.
TSA::ARMAspec(model = list(ar = phi))
title(main = expression(phi == 0.9))
ts.plot(arima.sim(n = 128, model = list(ar = phi)), ylab = "", main = "Simulated AR(1)")

# Negative AR coefficient: rapid alternation.
TSA::ARMAspec(model = list(ar = -phi))
title(main = expression(phi == -0.9))
ts.plot(arima.sim(n = 128, model = list(ar = -phi)), ylab = "", main = "Simulated AR(1)")
```



Spectral Density of an AR(2) Process with $\phi_1 = 1.5$ and $\phi_2 = -0.95$

An AR(2) process can concentrate power around an intermediate frequency. For these coefficients, the spectrum has a pronounced peak near 0.11 cycles per observation, corresponding to a cycle of roughly nine observations.

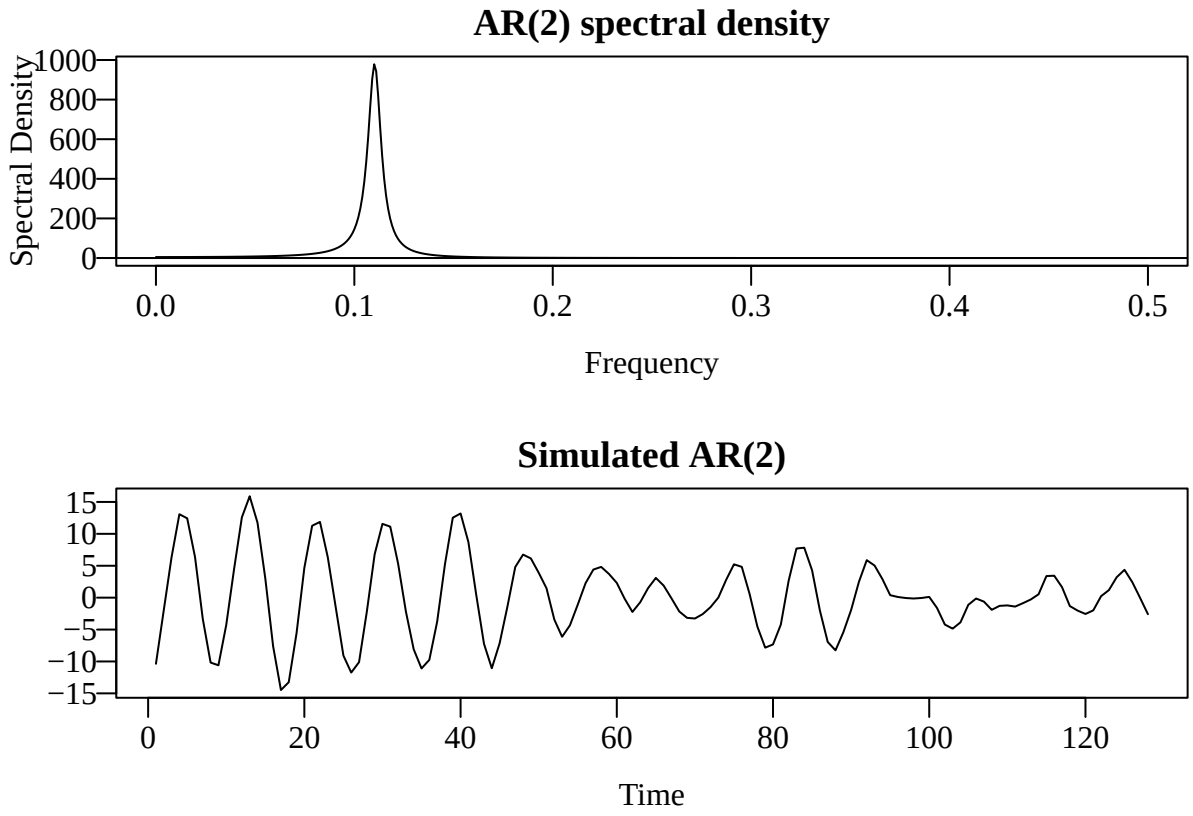
The simulated series exhibits stochastic oscillations: the timing and amplitude vary, unlike those of a deterministic cosine wave.

```r
phi <- c(1.5, -0.95)
```

```r
par(las = 1, mar = c(4, 4, 1.8, 0.6), mfrow = c(2, 1), mgp = c(2, 0.5, 0), family = "serif")

# The spectral peak identifies the dominant oscillation.
TSA::ARMAspec(model = list(ar = phi))
title(main = "AR(2) spectral density")

ts.plot(arima.sim(n = 128, model = list(ar = phi)), ylab = "", main = "Simulated AR(2)")
```



Spectral Density of a Seasonal AR Model

Consider the multiplicative seasonal model

$$(1 - 0.5B)(1 - 0.9B^{12})Y_t = \varepsilon_t.$$

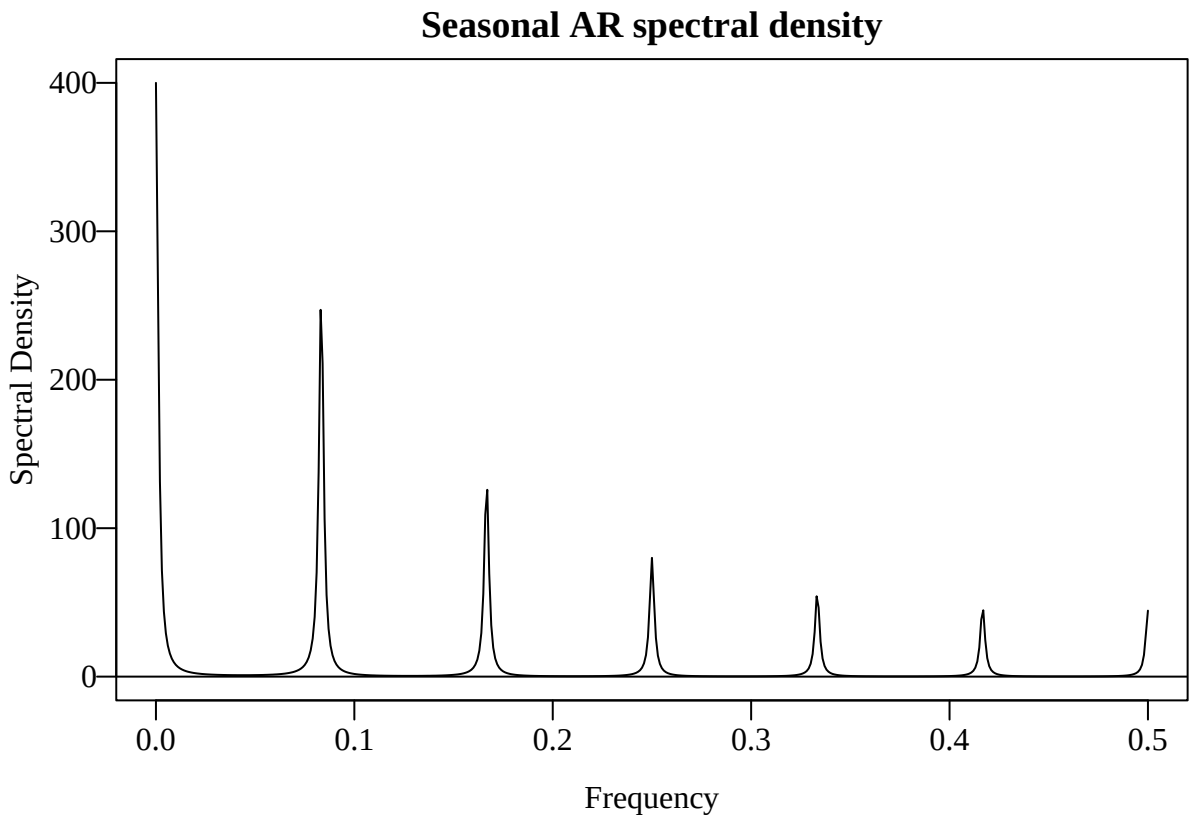
The seasonal AR term produces peaks at multiples of 1/12 cycles per observation. The nonseasonal AR term places additional emphasis on low frequencies, so the seasonal peaks need not have equal heights.

```r
phi <- 0.5
PHI <- 0.9

par(las = 1, mar = c(4, 4, 1.8, 0.6), mfrow = c(1, 1), mgp = c(2, 0.5, 0), family = "serif")

# Combine nonseasonal dependence with seasonality of period 12.
TSA::ARMAspec(
  model = list(ar = phi, seasonal = list(sar = PHI, period = 12))
  )
```

```
)
title(main = "Seasonal AR spectral density")
```



Spectral Estimation

The Periodogram Is Not a Consistent Estimator

The raw periodogram fluctuates substantially around the true spectral density. At an interior frequency, its standard deviation is approximately as large as its mean, even for a long series. Thus, increasing the sample size alone does not make the raw periodogram a consistent estimator.

We illustrate this behavior using repeated simulations from an AR(1) process with $\phi = -0.6$. Its spectral density is

$$f(\omega) = \frac{\sigma^2}{1 + \phi^2 - 2\phi\cos(2\pi\omega)}.$$

```
set.seed(123)

n <- 200
phi <- -0.6
N <- 100

# Simulate independent realizations from the same AR(1) model.
y <- replicate(N, arima.sim(model = list(ar = phi), n = n))

# Compute raw periodograms without tapering or detrending.
```

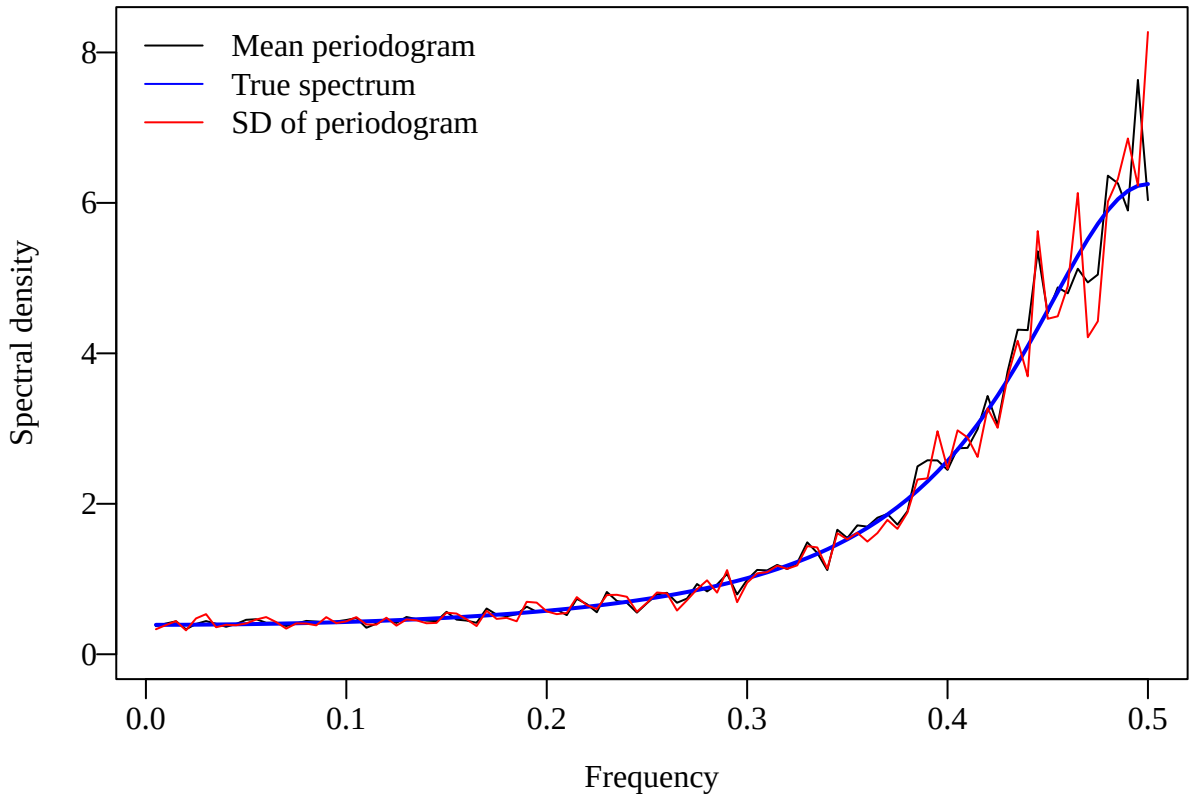
```r
raw_spec <- function(x) {
  spec.pgram(x, taper = 0, detrend = FALSE, demean = TRUE, fast = FALSE, plot = FALSE)
}

pg <- raw_spec(y[, 1])
freq <- pg$freq
spec_mat <- apply(y, 2, function(x) raw_spec(x)$spec)

# Compare the simulation mean and SD with the true spectrum.
true_spec <- 1 / (1 + phi^2 - 2 * phi * cos(2 * pi * freq))
mean_spec <- rowMeans(spec_mat)
sd_spec <- apply(spec_mat, 1, sd)

par(las = 1, mar = c(4, 4, 1, 0.6), mgp = c(2, 0.5, 0), mfrow = c(1, 1), family = "serif")

plot(freq, mean_spec, type = "l", ylim = range(0, mean_spec, sd_spec, true_spec),
     xlab = "Frequency", ylab = "Spectral density")
lines(freq, true_spec, col = "blue", lwd = 2)
lines(freq, sd_spec, col = "red")
legend("topleft",
       legend = c("Mean periodogram", "True spectrum", "SD of periodogram"),
       col = c("black", "blue", "red"), lty = 1, bty = "n")
```

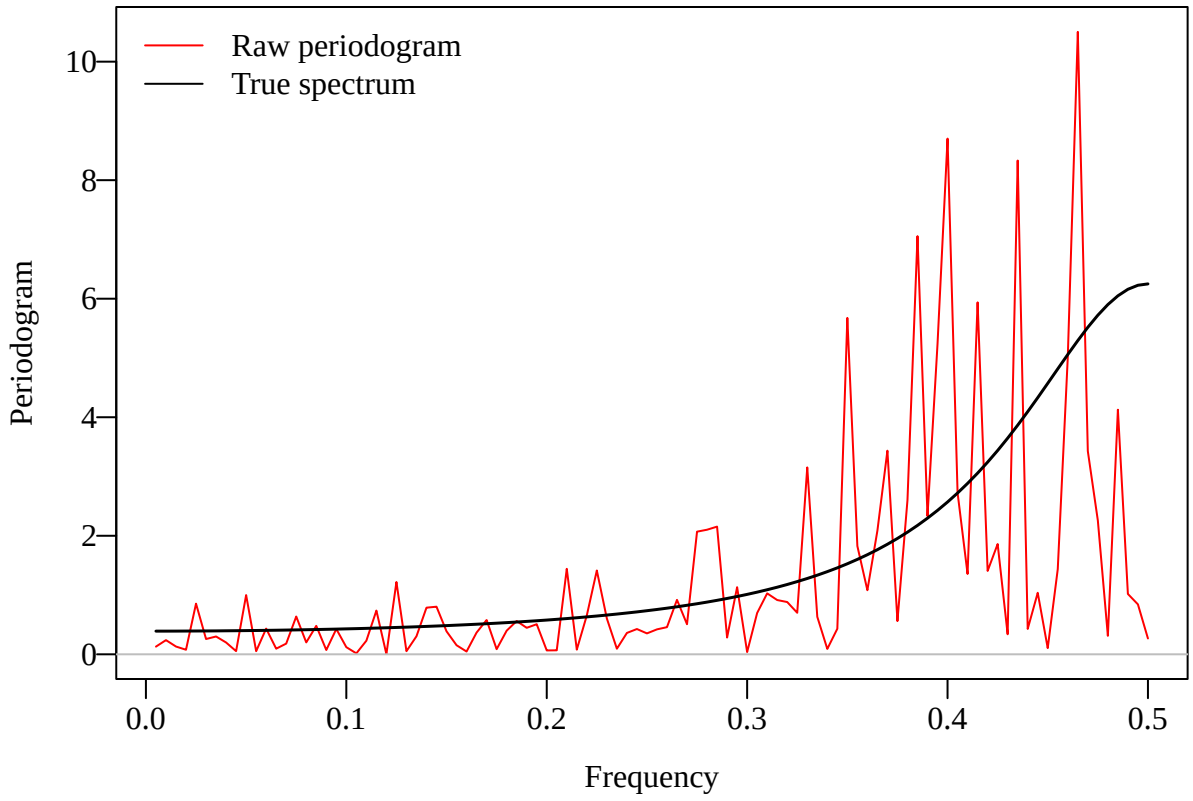


The average across independent realizations roughly follows the true spectrum, but the standard deviation remains substantial. This experiment illustrates the variability at one sample size; inconsistency concerns the persistence of that variability as the sample size increases. The approximation that the standard deviation equals the mean applies away from zero and the Nyquist frequency.

A single realization makes the variability particularly clear.

```r
sp <- raw_spec(y[, 1])

par(las = 1, mar = c(4, 4, 1, 0.6), mgp = c(2, 0.5, 0), family = "serif")
plot(sp$freq, sp$spec, type = "l", col = "red", xlab = "Frequency", ylab = "Periodogram")
lines(freq, true_spec, lwd = 1.5)
abline(h = 0, col = "gray")
legend("topleft", legend = c("Raw periodogram", "True spectrum"),
       col = c("red", "black"), lty = 1, bty = "n")
```



Averaged Periodogram

For a single observed series, we can reduce variability by averaging neighboring periodogram ordinates. A Daniell kernel with parameter m gives equal weight to $2m + 1$ neighboring ordinates.

A wider smoothing window reduces variance but can blur narrow peaks and introduce bias. For consistency, the number of averaged ordinates must increase with the sample size while the smoothing bandwidth in frequency units shrinks toward zero.

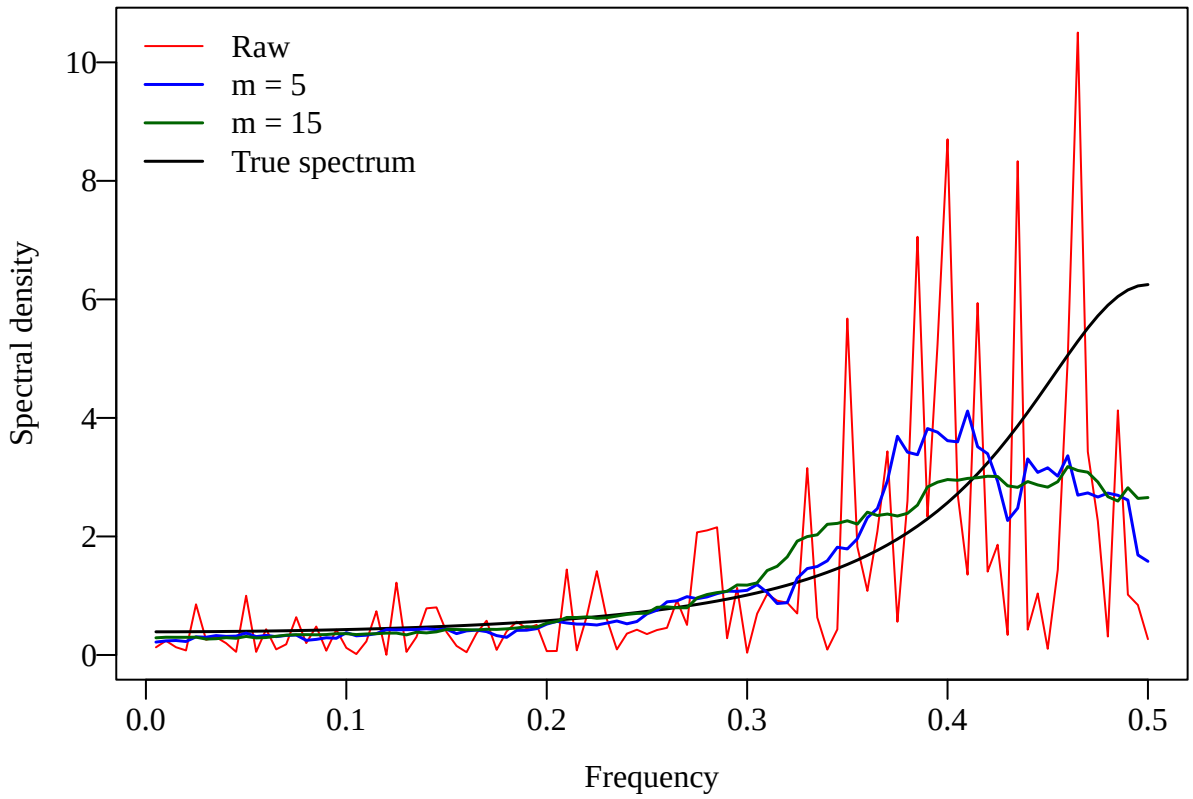
```r
# Smooth the same periodogram using two window widths.
sp_raw <- raw_spec(y[, 1])

sp5 <- spec.pgram(
  y[, 1], kernel = kernel("daniell", m = 5),
  taper = 0, detrend = FALSE, demean = TRUE,
  fast = FALSE, plot = FALSE
)
```

```r
sp15 <- spec.pgram(
  y[, 1], kernel = kernel("daniell", m = 15),
  taper = 0, detrend = FALSE, demean = TRUE,
  fast = FALSE, plot = FALSE
)

par(las = 1, mar = c(4, 4, 1, 0.6), mgp = c(2, 0.5, 0), family = "serif")
plot(sp_raw$freq, sp_raw$spec, type = "l", col = "red", xlab = "Frequency", ylab = "Spectral density")
lines(freq, true_spec, lwd = 1.5)
lines(sp5$freq, sp5$spec, col = "blue", lwd = 1.5)
lines(sp15$freq, sp15$spec, col = "darkgreen", lwd = 1.5)

legend("topleft",
       legend = c("Raw", "m = 5", "m = 15", "True spectrum"),
       col = c("red", "blue", "darkgreen", "black"),
       lty = 1, lwd = c(1, 1.5, 1.5, 1.5), bty = "n")
```



Pointwise Confidence Intervals for $f(\omega)$

An approximate confidence interval follows from

$$\frac{\nu \widehat{f}(\omega)}{f(\omega)} \approx \chi^2_\nu,$$

where ν is the effective degrees of freedom. An approximate 95% pointwise confidence interval is

$$\left[\frac{\nu \widehat{f}(\omega)}{\chi^2_{\nu,0.975}}, \frac{\nu \widehat{f}(\omega)}{\chi^2_{\nu,0.025}} \right].$$

These intervals describe uncertainty at individual frequencies; they do not provide simultaneous 95% coverage of the entire spectrum. Their accuracy also depends on the spectrum being approximately constant across the smoothing window, with additional care needed near the frequency boundaries.

```
sp <- sp15

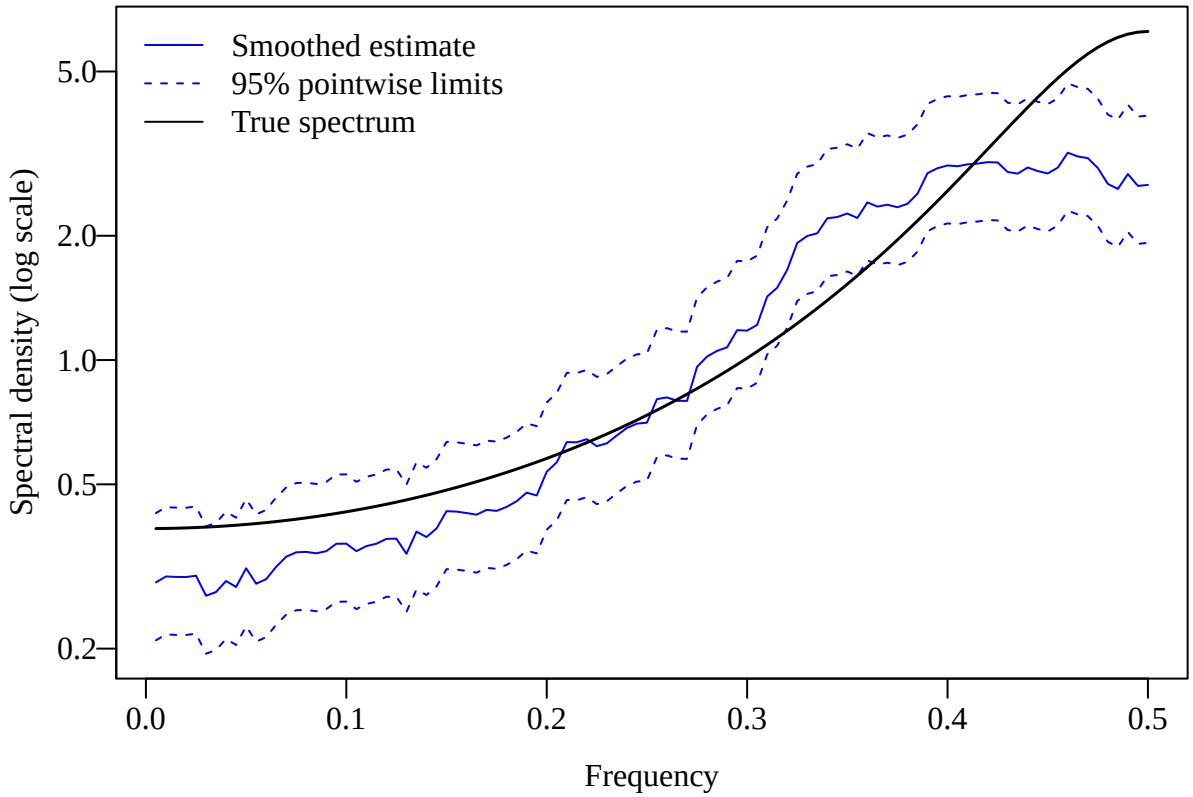
# Use the effective degrees of freedom returned by spec.pgram().
nu <- sp$df
lower <- nu * sp$spec / qchisq(0.975, df = nu)
upper <- nu * sp$spec / qchisq(0.025, df = nu)

true_spec <- 1 / (1 + phi^2 - 2 * phi * cos(2 * pi * sp$freq))

par(las = 1, mar = c(4, 4, 1, 0.6), mgp = c(2, 0.5, 0), family = "serif")

# Display the spectrum and intervals on a logarithmic y-axis.
plot(sp$freq, sp$spec, type = "l", log = "y", ylim = range(lower, upper, true_spec),
     xlab = "Frequency", ylab = "Spectral density (log scale)", col = "blue")
lines(sp$freq, lower, col = "blue", lty = 2)
lines(sp$freq, upper, col = "blue", lty = 2)
lines(sp$freq, true_spec, lwd = 1.5)

legend("topleft",
       legend = c("Smoothed estimate", "95% pointwise limits", "True spectrum"),
       col = c("blue", "blue", "black"),
       lty = c(1, 2, 1), bty = "n")
```

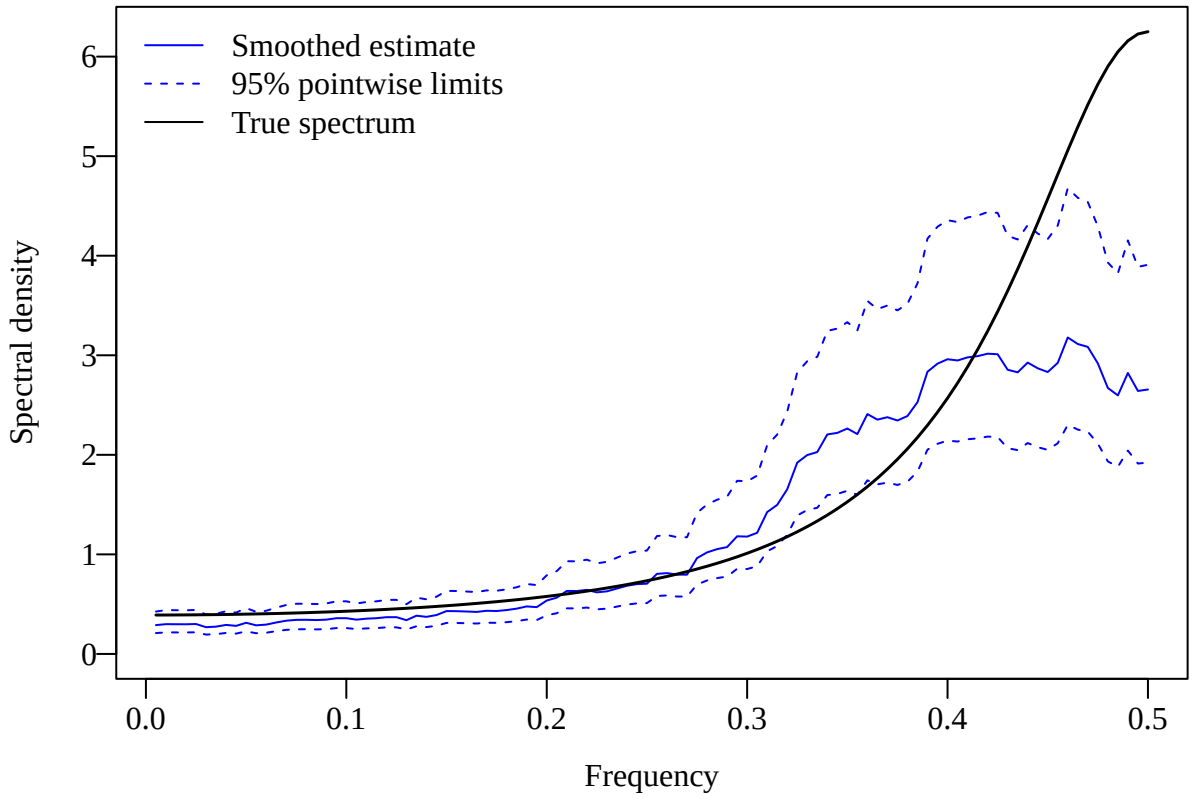


On a logarithmic axis, the confidence limits have a constant vertical separation from the estimate because they are fixed multiples of it. On the original scale, the intervals become wider where the estimated spectral

density is larger.

```
# Show the same intervals on the original scale.
par(las = 1, mar = c(4, 4, 1, 0.6), mgp = c(2, 0.5, 0), family = "serif")
plot(sp$freq, sp$spec, type = "l", ylim = range(0, lower, upper, true_spec),
     xlab = "Frequency", ylab = "Spectral density", col = "blue")
lines(sp$freq, lower, col = "blue", lty = 2)
lines(sp$freq, upper, col = "blue", lty = 2)
lines(sp$freq, true_spec, lwd = 1.5)

legend("topleft",
       legend = c("Smoothed estimate", "95% pointwise limits", "True spectrum"),
       col = c("blue", "blue", "black"),
       lty = c(1, 2, 1), bty = "n")
```



Leakage

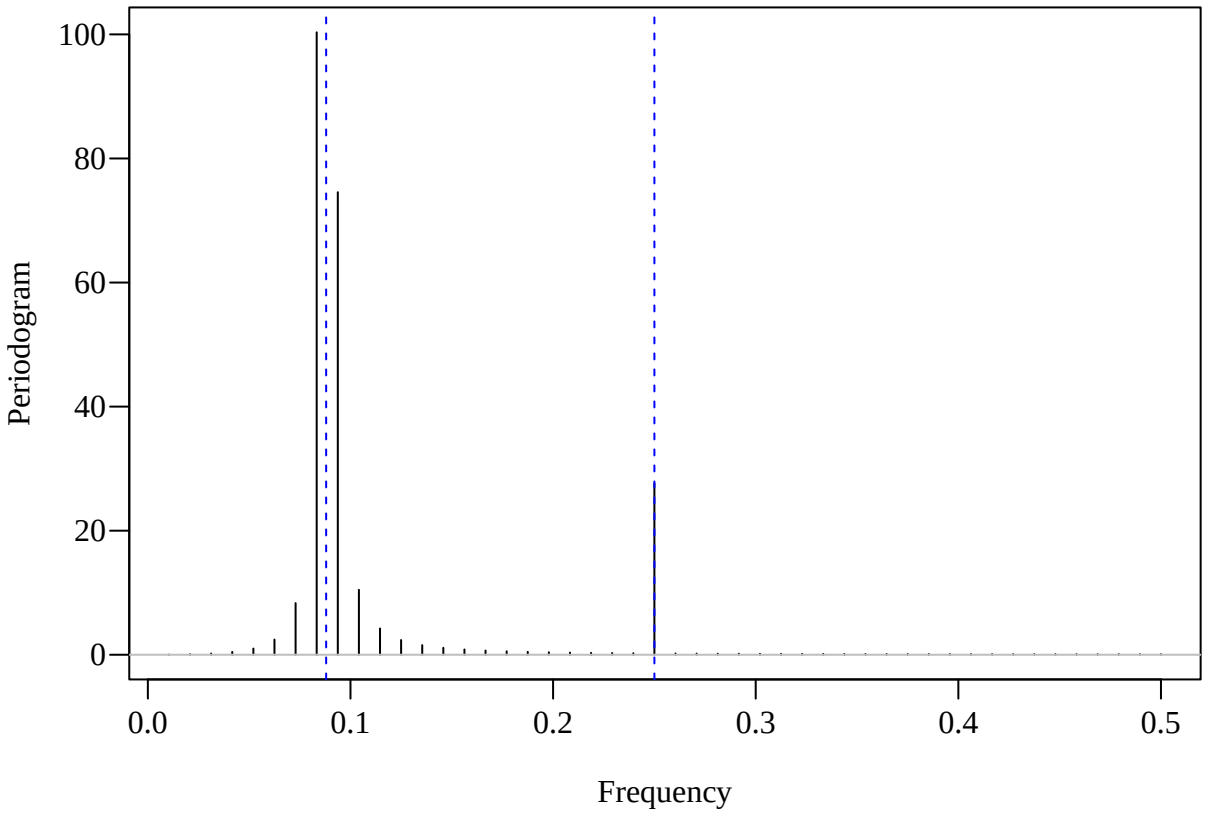
A finite observation span spreads the power of an oscillation across frequencies. On the Fourier grid, this spreading is especially apparent when the oscillation does not complete an integer number of cycles within the observed record.

Here, 0.088 does not lie on the Fourier grid, whereas $24/96 = 0.25$ does. The first component therefore shows more visible spreading across neighboring Fourier frequencies.

```
n <- 96
t <- seq_len(n)
f1 <- 0.088
f2 <- 24 / n
```

```r
# Combine an off-grid frequency and an on-grid frequency.
y_leak <- 3 * cos(2 * pi * f1 * t) + sin(2 * pi * f2 * t)
sp_leak <- raw_spec(y_leak)

par(las = 1, mar = c(4, 4, 1, 0.6), mgp = c(2.4, 0.6, 0), family = "serif")
plot(sp_leak$freq, sp_leak$spec, type = "h", xlab = "Frequency", ylab = "Periodogram")
abline(h = 0, col = "gray")
abline(v = c(f1, f2), col = "blue", lty = 2)
```



Tapering

Tapering gradually downweights observations near the ends of the record. It reduces sidelobes and leakage into distant frequencies, but broadens the main spectral peak and can make nearby oscillations harder to distinguish.

First, we visualize a split cosine-bell taper and its effect on a series. In `spec.taper()`, `p` is the proportion tapered at each end; `p = 0.5` tapers the entire record.

```r
set.seed(456)

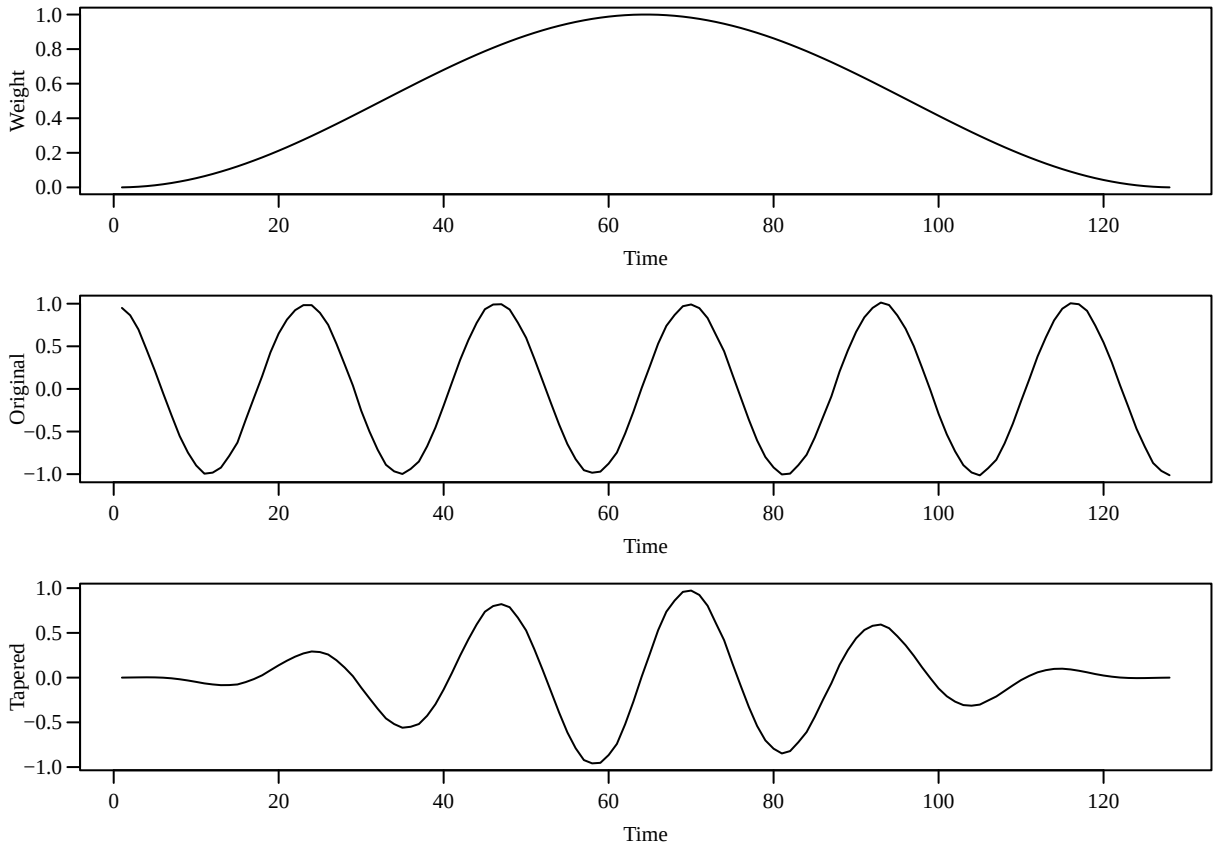
n <- 128
t <- seq_len(n)

# Use an off-grid oscillation to make leakage visible.
x5h <- cos(2 * pi * (5.5 / n) * t) + rnorm(n, sd = 0.01)
```

```r
taper_weights <- as.numeric(spec.taper(rep(1, n), p = 0.5))
x_tapered <- as.numeric(spec.taper(x5h, p = 0.5))

par(mfrow = c(3, 1), mar = c(3, 4, 1, 0.6), mgp = c(2, 0.7, 0), las = 1, family = "serif")

plot(t, taper_weights, type = "l", xlab = "Time", ylab = "Weight")
plot(t, x5h, type = "l", xlab = "Time", ylab = "Original")
plot(t, x_tapered, type = "l", xlab = "Time", ylab = "Tapered")
```

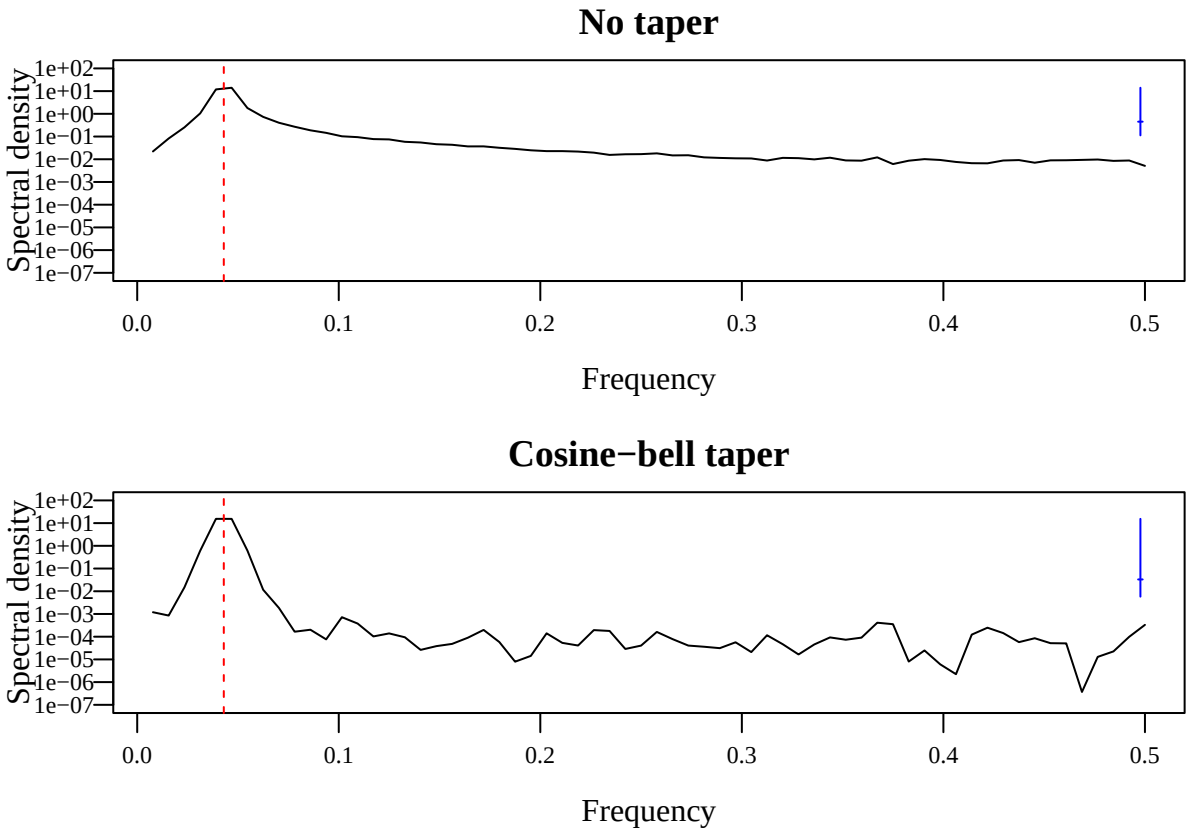


Next, we compare the untapered and tapered periodograms on the same logarithmic scale. The taper suppresses distant leakage while spreading power over a wider region near the generating frequency.

```r
par(mfrow = c(2, 1), mar = c(3.5, 4, 2, 0.6), mgp = c(2, 0.5, 0), las = 1, family = "serif")

# Untapered periodogram.
spec.pgram(x5h, taper = 0, detrend = FALSE, demean = TRUE,
           fast = FALSE, log = "yes", ylim = c(1e-7, 1e2),
           main = "No taper", sub = "",
           xlab = "Frequency", ylab = "Spectral density", cex.axis = 0.75)
abline(v = 5.5 / n, col = "red", lty = 2)

# Tapered periodogram, with no frequency smoothing.
spec.pgram(x5h, taper = 0.5, detrend = FALSE, demean = TRUE,
           fast = FALSE, log = "yes", ylim = c(1e-7, 1e2),
           main = "Cosine-bell taper", sub = "",
           xlab = "Frequency", ylab = "Spectral density", cex.axis = 0.75)
abline(v = 5.5 / n, col = "red", lty = 2)
```




```
par(mfrow = c(1, 1))
```

Smoothing and tapering serve different purposes: smoothing reduces the sampling variability of spectral estimates, whereas tapering controls leakage caused by the finite observation span.

References

Cryer, Jonathan D, and Kung-Sik Chan. 2008. *Time Series Analysis: With Applications in r*. Vol. 2. Springer.

Shumway, Robert H, and David S Stoffer. 2017. *Time Series Analysis and Its Applications*. 4th ed. Springer.